

Technical Abstract

Multi-Source Candidate Data Transformer

Unified Candidate Record Pipeline Across CSV, JSON, GitHub and Resume Sources

Overview

The system ingests heterogeneous candidate information from CSV exports, JSON payloads, GitHub API profiles and resume PDFs, transforming them into a unified canonical candidate record. A seven-stage pipeline performs source detection, extraction, normalization, entity resolution, confidence scoring, configuration-driven projection and schema validation to generate a consistent and explainable output irrespective of source combination or data volume.

Pipeline Stages

Stage	Description
Ingest & Detect	Identify source type (CSV, JSON, GitHub API or Resume PDF).
Extract	Parse source-specific inputs into a standard RawRecord .
Normalize	Standardize dates, phone numbers, countries and textual fields.
Entity Resolution	Merge duplicate candidate records and resolve conflicts.
Confidence Scoring	Assign explainable confidence scores to every field.
Projection	Apply configuration-based field mapping and filtering.
Validation	Validate output against dynamically generated JSON schema.

Normalization

Phones are converted to E.164 using the **phonenumbers** library, **Dates** are normalized to YYYY-MM using **dateparser**, **Countries** are mapped to ISO-3166 alpha-2 codes, while **Skills** are canonicalized using a synonym dictionary. Names and emails are lowercased and whitespace normalized before entity matching.

Merge Strategy

Email acts as the primary identifier, with *normalized full name + phone number* used as a fallback. Conflicts follow the source priority

ATS > Resume > GitHub > CSV

while recent timestamps determine employment information and majority voting determines skill values.

Confidence Model

Field confidence is computed using

$$\text{Confidence} = \text{Base Score} + \text{Agreement Bonus} - \text{Staleness Penalty}$$

Component	Description
Base Score	Reliability assigned to each source.
Agreement Bonus	Added when multiple sources agree.
Staleness Penalty	Reduced according to data age.

Runtime Projection

The canonical record is maintained independently of the projection layer. Runtime configuration determines field selection, remapping, null handling and schema validation before the final output is emitted.

Edge Cases

- Invalid CSV rows are skipped with warnings.
- Empty GitHub repositories degrade gracefully.
- Company conflicts follow source-priority ranking.
- Unparseable dates return **null**.
- Alternate spellings are retained as aliases.
- Scanned-PDF OCR and LinkedIn scraping are intentionally out of scope.